机器学习开卷考试复习资料 - SVM与神经网络

快速索引

按任务分类

- **分类任务**: 最大间隔分类器 (MMC)、支持向量分类器 (SVC)、支持向量机 (SVM)、前馈神经网络 (FNN)、卷积神经网络 (CNN)、循环神经网络 (RNN)、长短期记忆网络 (LSTM)
- **回归任务**: 前馈神经网络 (FNN)
- **非线性分类**: 支持向量机 (SVM)、深度神经网络 (DNN)

缩写对照表（A-Z排序）

- **ANN**: Artificial Neural Network / 人工神经网络
- **CNN**: Convolutional Neural Network / 卷积神经网络
- **DNN**: Deep Neural Network / 深度神经网络
- **FNN**: Feedforward Neural Network / 前馈神经网络
- **LSTM**: Long Short-Term Memory / 长短期记忆网络
- **MMC**: Maximal Margin Classifier / 最大间隔分类器
- **MLP**: Multilayer Perceptron / 多层感知机
- **NN**: Neural Network / 神经网络
- **ReLU**: Rectified Linear Unit / 修正线性单元
- **RNN**: Recurrent Neural Network / 循环神经网络
- **SVC**: Support Vector Classifier / 支持向量分类器
- **SVM**: Support Vector Machine / 支持向量机

详细方法总结

方法1: 超平面 / Hyperplane

缩写: 无

页码: ML-lec-6-SVM.pdf p.10-12

解决问题:

- 用于分割p维特征空间
- 将空间分为两个半空间
- 是分类器的决策边界

工作原理:

1. 在二维空间中，超平面是一条直线，方程为 $\beta_0 + \beta_1X_1 + \beta_2X_2 = 0$
2. 在p维空间中，超平面方程为 $\beta_0 + \beta_1X_1 + \beta_2X_2 + ... + \beta_pX_p = 0$
3. 超平面将p维空间分成两部分：$\beta_0 + \beta_1X_1 + ... + \beta_pX_p > 0$ 和 $\beta_0 + \beta_1X_1 + ... + \beta_pX_p < 0$
4. 测试观测根据落在超平面的哪一侧来分类

重要公式/概念:

- 超平面定义: $\beta_0 + \beta_1X_1 + \beta_2X_2 + ... + \beta_pX_p = 0$

- 分类函数: $f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2^* + \ldots + \beta_p x_p^*$
- $|f(x^*)|$的大小表示点到超平面的距离，也反映分类的置信度

关键要点:

- 参数$\beta_0, \beta_1, \ldots, \beta_p$决定超平面的位置和方向
- 超平面是线性分类器的基础
- $f(x^*)$越大，分类置信度越高

方法1: K均值聚类 / K-Means Clustering

缩写: K-Means

页码: ML-final review.pdf p.168-173

解决问题:

用于将数据样本划分为K个互不相交的簇（子组），使得同一簇内的样本尽可能相似，不同簇的样本尽可能不同。

工作原理:

1. 随机初始化K个簇中心（质心）。
2. 重复以下步骤直到收敛（簇分配不再改变）：
3. a. 将每个样本分配到距离其最近的质心所在的簇（使用欧氏距离）。
4. b. 重新计算每个簇的质心（即该簇所有样本特征的平均值向量）。
5. 输出最终的簇划分和质心。
6. 重要公式/概念:
    - 簇内差异：$W(C_k)$ 衡量簇 C_k 内样本的离散程度。
    - 目标函数：最小化总簇内差异
    - 欧氏距离：用于度量样本与质心之间的距离。
    - 关键要点:
    - 需要预先指定簇数K。
    - 对初始质心敏感，可能陷入局部最优。
    - 适用于球形簇，对异常值敏感。
    - 收敛速度快，但可能停止在局部最优解。

方法2: 主成分分析 / Principal Component Analysis

缩写: PCA

页码: ML-final review.pdf p.161-167

解决问题:

用于数据降维、可视化和特征提取，通过线性变换将原始特征转换为一组线性不相关的主成分，保留数据中的主要方差。

工作原理:

1. 对原始数据进行标准化处理（通常使均值为0，方差为1）。
2. 计算数据的协方差矩阵（或相关矩阵）。
3. 计算协方差矩阵的特征值和特征向量。
4. 将特征值从大到小排序，选择前K个最大特征值对应的特征向量作为主成分方向。
5. 将原始数据投影到选定的主成分方向上，得到降维后的新数据。
6. 重要公式/概念:
    - 第一主成分：$Z_1 = \phi_{11} X_1 + \phi_{21} X_2 + \cdots + \phi_{p1} X_p$，其中 ϕ_1 是使得样本方差最大的单位向量。
    - 方差解释比例：$PVE_i = \frac{\hat{\sigma}_{zi}^2}{\hat{\tau}V}$，表示第i个主成分解释的方差占总方差的比例。

- 总方差：$\hat{TV} = \sum_{j=1}^{p} \hat{\sigma}_{x_j}^2$。
- 关键要点:
- 主成分是原始特征的线性组合，且彼此正交（不相关）。
- 第一主成分方向是数据方差最大的方向，后续主成分依次在与之前主成分正交的方向中选取方差最大的方向。
- 可用于数据可视化（降至2-3维）、噪声过滤和特征提取。
- 是一种无监督方法，不需要标签信息。

方法3: 强化学习 / Reinforcement Learning

缩写: RL

页码: ML-final review.pdf p.180-186

解决问题:

用于解决序列决策问题，智能体通过与环境的交互学习最优策略，以最大化累积奖励。适用于游戏、机器人控制、自动驾驶等领域。

工作原理:

1. 智能体在时间步t观测环境状态 S_t，并执行动作 A_t。
2. 环境根据动作转移到新状态 S_{t+1}，并给出即时奖励 R_{t+1}。
3. 智能体根据奖励和状态更新其策略（动作选择规则），目标是最大化长期累积奖励。
4. 通过试错（探索与利用的权衡）不断优化策略。
5. 重要公式/概念:
    - 状态（State）：对环境当前情况的完整描述。
    - 动作（Action）：智能体可执行的操作。
    - 奖励（Reward）：环境对智能体动作的即时反馈信号。
    - 策略（Policy）：状态到动作的映射函数，决定智能体的行为。
    - 价值函数（Value Function）：评估在给定状态或状态下执行给定动作的长期价值。
    - 关键要点:
    - 核心是学习一个最大化累积奖励的策略。
    - 需要平衡探索（尝试新动作以发现更高奖励）和利用（选择已知高奖励的动作）。
    - 通常使用马尔可夫决策过程（MDP）建模。
    - 无需带标签的训练数据，通过与环境的交互自主学习。
    - 主要算法包括Q-learning、策略梯度、深度强化学习（如DQN）等。

方法2: 最大间隔分类器 / Maximal Margin Classifier (MMC)

缩写: MMC, Optimal Separating Hyperplane

页码: ML-lec-6-SVM.pdf p.20-30

解决问题:

- 用于线性可分的二分类问题
- 在所有可能的分割超平面中选择最优的一个
- 要求数据完全线性可分（硬间隔）

工作原理:

1. **间隔 (Margin)**: 计算每个观测到超平面的垂直距离，最小距离即为间隔
2. **最大间隔超平面**: 选择使间隔最大的超平面
3. **支持向量 (Support Vectors)**: 落在间隔边界上的训练观测点，决定超平面位置
4. 分类决策: 根据测试点落在超平面哪一侧判断类别

重要公式/概念:

- 优化目标: maximize M，subject to $\Sigma\beta_j^2 = 1$
- 约束条件: $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip}) \geq M$，$\forall i = 1,...,n$
- 第i个观测到超平面的垂直距离: $y_i(\beta_0 + \beta_1 x_{i1} + ... + \beta_p x_{ip})$
- 分类函数: $f(x) = \beta_0 + \beta_1 x_1 + ... + \beta_p x_p$*

关键要点:

- 只由支持向量决定，对其他点的位置不敏感
- 仅适用于完全线性可分数据
- 移动非支持向量不影响分类器
- 希望训练集的大间隔在测试集上也保持

方法3: 支持向量分类器 / Support Vector Classifier (SVC)

缩写: SVC, Soft Margin Classifier
页码: ML-lec-6-SVM.pdf p.31-44

解决问题:

- 用于线性不可分或近似线性可分的二分类问题
- 允许一定程度的误分类（软间隔）
- 比最大间隔分类器更具鲁棒性和泛化能力

工作原理:

1. **松弛变量 (Slack Variable) ε_i**: 允许观测违反间隔甚至超平面
    - $\varepsilon_i = 0$: 观测在间隔正确一侧
    - $0 < \varepsilon_i \leq 1$: 观测在间隔错误一侧，但在超平面正确一侧
    - $\varepsilon_i > 1$: 观测在超平面错误一侧（误分类）
2. **调节参数C**: 控制对违反间隔的容忍度
3. C越大，间隔越宽，允许更多违反；C越小，间隔越窄，更严格
4. 通过交叉验证选择最优C值

重要公式/概念:

- 优化目标: maximize M，subject to $\Sigma\beta_j^2 = 1$
- 约束条件: $y_i(\beta_0 + \beta_1 x_{i1} + ... + \beta_p x_{ip}) \geq M(1 - \varepsilon_i)$
- 附加约束: $\varepsilon_i \geq 0$，$\Sigma\varepsilon_i \leq C$
- C是所有ε_i之和的上界

关键要点:

- C是关键超参数，控制偏差-方差权衡
- C小: 低偏差，高方差，间隔窄，对训练数据敏感
- C大: 高偏差，低方差，间隔宽，更多误分类

- 通过validation　set或交叉验证选择C

方法4: 支持向量机 / Support Vector Machine (SVM)

缩写: SVM

页码: ML-lec-6-SVM.pdf p.45-52

解决问题:

- 用于非线性可分的分类问题
- 通过核函数将数据映射到高维空间，实现非线性决策边界
- 适用于复杂的分类任务

工作原理:

1. **核函数 (Kernel)**: 度量两个观测之间相似性的函数
2. **线性核**: $K(x_i, x_i') = \Sigma x_{ij}x_i'_j$（即支持向量分类器）
3. **多项式核**: $K(x_i, x_i') = (1 + \Sigma x_{ij}x_i'_j)^d$（d阶多项式）
4. **径向核 (RBF)**: $K(x_i, x_i') = \exp(-\gamma\Sigma(x_{ij} - x_i'_j)^2)$
5. 分类函数表示为: $f(x) = \beta_0 + \Sigma\alpha_i \cdot K(x, x_i)$

重要公式/概念:

- 线性核: $K(x_i, x_i') = \Sigma x_{ij}x_i'_j$
- d阶多项式核: $K(x_i, x_i') = (1 + \Sigma x_{ij}x_i'_j)^d$
- 径向核: $K(x_i, x_i') = \exp(-\Sigma(x_{ij} - x_i'_j)^2)$
- SVM = 支持向量分类器 + 非线性核函数

关键要点:

- 核技巧避免显式计算高维特征
- 多项式核适合具有多项式关系的数据
- 径向核最常用，可产生局部性很强的决策边界
- 核函数的选择取决于具体问题

Method	Definition	Note
最大间隔分类器（硬间隔）	找到一个完全线性可分数据集的最大间隔超平面，并确保所有样本都正确分类。	仅考虑硬间隔分类，不允许错误分类。
支持向量分类器（软间隔）	允许错误分类的存在，引入松弛变量来容忍一定的误分类。	软间隔分类器
支持向量机（软间隔）	支持向量分类器的扩展，通过核函数扩大特征空间。	可处理非线性问题

方法5: 人工神经元 / Artificial Neuron

缩写: 无

页码: ML-lec-7-NN.pdf p.14-18

解决问题:

- 神经网络的基本计算单元
- 模拟生物神经元的信息处理过程
- 接收输入，进行加权求和，通过激活函数产生输出

工作原理:

1. **输入 (Input) X**: 来自上一层神经元或原始特征
2. **权重 (Weight) W**: 表示每个输入的重要程度
3. **偏置 (Bias) b**: 激活神经元的阈值
4. **加权求和**: $z = \Sigma w_i x_i + b = w^T x + b$
5. **激活函数**: $y = f(z)$，引入非线性

重要公式/概念:

- 输入信号加权汇总: $z = \Sigma w_i x_i + b = w^T x + b$
- 输出: $y = f(z)$
- 模拟生物神经元: 树突(输入) → 细胞体(处理) → 轴突(输出)

关键要点:

- 权重w决定输入的重要性
- 偏置b控制神经元激活的难易程度
- 激活函数引入非线性，使网络能学习复杂模式
- 单个神经元只能实现线性分类

方法6: 激活函数 / Activation Function

缩写: 无
页码: ML-lec-7-NN.pdf p.25-34

解决问题:

- 为神经网络引入非线性
- 控制神经元的激活状态
- 不同激活函数适用于不同场景

工作原理:

1. **Sigmoid函数**: $\sigma(x) = 1/(1 + \exp(-x))$，输出范围(0,1)
2. **Tanh函数**: $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$，输出范围(-1,1)
3. **ReLU函数**: $\mathrm{ReLU}(x) = \max(0, x)$，最常用
4. **Leaky ReLU**: $f(x) = 0.01x$ ($x<0$) 或 x ($x\geq 0$)

重要公式/概念:

- Sigmoid: $\sigma(x) = 1/(1 + \exp(-x))$，梯度范围(0, 0.25)
- Tanh: $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$
- ReLU: $\mathrm{ReLU}(x) = \max(0, x)$
- Leaky ReLU: 避免神经元死亡问题

关键要点:

- **Sigmoid/Tanh问题**: 梯度消失（梯度范围小）、计算复杂
- **Sigmoid/Tanh优点**: 输出范围可控、不存在dead neuron
- **ReLU优点**: 计算简单、缓解梯度消失
- **ReLU问题**: Dead Neuron（神经元永久输出0）
- **Leaky ReLU**: 解决ReLU的Dead Neuron问题

方法7: 前馈神经网络 / Feedforward Neural Network (FNN)

缩写: FNN，Multilayer Perceptron（MLP）

页码: ML-lec-7-NN.pdf p.50-54

解决问题:

- 用于分类和回归任务
- 通过多层神经元学习复杂的非线性映射
- 网络中无环路，信息单向传播

工作原理:

1. **输入层**: 接收外部数据，神经元个数=特征数p
2. **隐藏层**: 中间处理层，可以有多层
3. **输出层**: 输出最终结果，神经元个数=目标变量数q
4. **前向传播**: x = a^(0) → z^(1) → a^(1) → z^(2) → ... → a^(L)
5. 第l层的计算: z^(l) = W^(l)·a^(l-1) + b^(l)，a^(l) = f_l(z^(l))

重要公式/概念:

- 第l层输入: z^(l) = W^(l)·a^(l-1) + b^(l)
- 第l层输出: a^(l) = f_l(z^(l))
- L: 隐藏层层数
- M_l: 第l层神经元个数
- W^(l): 第l-1到第l层的权重矩阵
- b^(l): 第l-1到第l层的偏置

关键要点:

- 层数和神经元数决定模型复杂度
- 同层神经元之间无连接
- 不存在跨层连接
- 通过validation set确定网络结构

方法8: 梯度下降 / Gradient Descent

缩写: GD

页码: ML-lec-7-NN.pdf p.55-57

解决问题:

- 神经网络参数优化算法
- 通过迭代更新找到损失函数的最小值

- 训练FNN的核心方法

工作原理:

1. 随机初始化参数 $\beta = (\beta_1, \beta_2, ..., \beta_m)$
2. 计算损失函数 L(w)，如MSE: $L(w) = (1/n)\Sigma(f_w(x_i) - y_i)^2$
3. 计算梯度 $\partial L(w)/\partial w$
4. 更新参数: w_{t+1} = w_t - α·(∂L(w)/∂w_t)
5. 重复步骤2-4直到收敛

重要公式/概念:

- MSE损失: $L(w) = (1/n)\Sigma(f_w(x_i) - y_i)^2$
- 参数更新: w_{t+1} = w_t - α·(∂L/∂w_t)
- 学习率α: 控制更新步长
- 同时更新所有参数

关键要点:

- α是学习率（如0.1），控制更新步长
- α太大: 可能不收敛；α太小: 收敛太慢
- 需要计算损失函数对所有参数的偏导数
- 通过反向传播(backpropagation)高效计算梯度

方法9: 反向传播 / Backpropagation

缩写: BP

页码: ML-lec-7-NN.pdf p.13-23

解决问题:

- 高效计算神经网络中损失函数对每个参数的梯度
- 训练深度神经网络的关键算法
- 使用链式法则逐层计算梯度

工作原理:

1. **前向传播**: 计算网络输出和损失
2. **反向传播**: 从输出层向输入层逐层计算梯度
3. 使用链式法则: $\partial J/\partial w_2 = (\partial J/\partial f_4)\cdot(\partial f_4/\partial f_3)\cdot(\partial f_3/\partial w_2)$
4. 更新参数: w = w - α·(∂J/∂w)
5. **梯度消失问题**: 在深层网络中，梯度可能指数级衰减
6. **梯度爆炸问题**: 权重初始化不当或网络太深，梯度可能指数级增长

重要公式/概念:

- 链式法则: $\partial J/\partial w_0 = (\partial J/\partial f_4)\cdot(\partial f_4/\partial f_3)\cdot(\partial f_3/\partial f_2)\cdot(\partial f_2/\partial f_1)\cdot(\partial f_1/\partial w_0)$
- 权重更新: w_ij^(l) = w_ij^(l) - α·(∂J/∂w_ij^(l))
- 偏置更新: b^(l) = b^(l) - α·(∂J/∂b^(l))

关键要点:

- **梯度消失**: 使用Sigmoid/Tanh时，浅层梯度接近0，导致参数几乎不更新
- **梯度爆炸**: 权重初始化过大或网络太深，梯度变得极大

- **解决方案**：使用ReLU激活函数、合理初始化权重、使用残差连接
- 训练深层网络的核心挑战

方法10: Softmax函数 / Softmax Function

缩写: 无
页码: ML-lec-7-NN.pdf p.28

解决问题:

- 将神经网络输出转换为概率分布
- 用于多分类问题的输出层
- 保证所有类别概率之和为1

工作原理:

1. 输入: 神经网络最后一层的输出 $y_1, y_2, ..., y_K$
2. 对每个输出应用指数函数并归一化
3. 输出: $\theta_i = \exp(y_i) / \Sigma\exp(y_j)$，其中$i = 1, 2, ..., K$
4. θ_i表示样本属于第i类的概率
5. 满足: $\theta_i \in [0,1]$ 且 $\Sigma\theta_i = 1$

重要公式/概念:

- Softmax: $\theta_i = \exp(y_i) / \Sigma\exp(y_j)$
- 概率性质: $0 \leq \theta_i \leq 1$，$\Sigma\theta_i = 1$
- 与交叉熵损失函数配合使用

关键要点:

- 用于多分类问题的输出层
- 输出可解释为概率
- 预测类别: 选择概率最大的类
- 训练时配合交叉熵损失函数

方法11: 深度神经网络 / Deep Neural Network (DNN)

缩写: DNN
页码: ML-lec-7-NN.pdf p.7-9

解决问题:

- 学习复杂的层次化特征表示
- 处理大规模、高维数据
- 广泛应用于计算机视觉、自然语言处理等领域

工作原理:

1. **多层结构**：具有多个隐藏层的神经网络（一般≥3层）
2. **层次化特征学习**：浅层学习简单特征，深层学习抽象特征
3. 包括多种架构: MLP（全连接）、CNN（卷积）、RNN（循环）、Transformer等
4. 通过反向传播和梯度下降训练
5. 层数越多，表达能力越强，但训练难度越大

重要公式/概念:

- DNN形式: y = f_L(f_{L-1}(...f_2(f_1(x))))
- 深度 = 隐藏层数
- 每层可以使用不同的激活函数和结构

关键要点:

- 深度网络可以学习层次化表示
- 面临梯度消失/爆炸问题
- 需要大量数据和计算资源
- 不同任务选择不同架构（CNN用于图像，RNN用于序列）

方法12: 卷积神经网络 / Convolutional Neural Network (CNN)

缩写: CNN
页码: ML-lec-7-NN.pdf p.30-51

解决问题:

- 专门用于处理网格结构数据（如图像）
- 解决全连接网络的参数过多、丢失空间信息等问题
- 具有平移不变性和局部连接特性

工作原理:

1. **卷积层 (Convolution)**: 使用卷积核提取局部特征
    - 卷积核在输入上滑动，计算加权和
    - 参数共享，大幅减少参数量
2. **池化层 (Pooling)**: 下采样，减少特征图尺寸
    - Max Pooling: 取区域内最大值
    - Average Pooling: 取区域内平均值
3. **全连接层**: 最后进行分类或回归
4. 典型结构: 卷积层 → 激活 → 池化 → ... → 全连接层

重要公式/概念:

- **卷积操作**: 输出 = Σ(输入区域 × 卷积核) + 偏置
- **输出尺寸**: O = (W - K + 2P)/S + 1
    - W: 输入尺寸, K: 卷积核大小, P: Padding, S: Stride
- **Kernel Size**: 卷积核大小（如3×3）
- **Stride**: 步长，卷积核滑动距离
- **Padding**: 边缘填充0，控制输出尺寸

关键要点:

- 参数共享: 同一卷积核在整个输入上共享参数
- 局部连接: 每个神经元只连接输入的局部区域
- 平移不变性: 对图像位移具有鲁棒性
- 典型应用: 图像分类、目标检测、图像分割
- Max Pooling保留最显著特征，Average Pooling保留整体信息

方法13: 循环神经网络 / Recurrent Neural Network (RNN)

缩写: RNN

页码: ML-lec-7-NN.pdf p.53-66

解决问题:

- 处理序列数据（文本、语音、时间序列等）
- 捕捉序列中的时序依赖关系
- 解决FNN无法处理变长输入和上下文信息的问题

工作原理:

1. **内部状态 (Hidden State)**: 保存历史信息
2. 每个时间步t的计算:
    - 隐状态更新: s_t = f(U·x_t + W·s_{t-1})
    - 输出: o_t = g(V·s_t)
3. **参数共享**: 所有时间步共享参数U, W, V
4. U: 输入到隐状态的权重
5. W: 隐状态到隐状态的权重（循环权重）
6. V: 隐状态到输出的权重

重要公式/概念:

- 隐状态更新: s_t = f(U·x_t + W·s_{t-1})
- 输出: o_t = g(V·s_t)
- 参数: U（输入权重）、W（循环权重）、V（输出权重）
- 所有时间步共享相同参数

关键要点:

- 能处理变长序列
- 考虑上下文信息（如"apple"在不同句子中的含义）
- **梯度消失问题严重**: 长序列中，梯度指数级衰减，难以学习长期依赖
- **梯度爆炸问题**: 梯度可能指数级增长
- 解决方案: 使用LSTM或GRU

方法14: 长短期记忆网络 / Long Short-Term Memory (LSTM)

缩写: LSTM

页码: ML-lec-7-NN.pdf p.68-74

解决问题:

- 解决RNN的梯度消失问题
- 能够学习长期依赖关系
- 通过门控机制选择性记忆和遗忘信息

工作原理:

1. **细胞状态 (Cell State) C_t**: 信息的高速通道，贯穿整个链
2. **遗忘门 (Forget Gate)**: 决定丢弃哪些信息
    - f_t = σ(W_f·[h_{t-1}, x_t] + b_f)

3. **输入门 (Input Gate)**: 决定更新哪些信息
    - i_t = σ(W_i·[h_{t-1}, x_t] + b_i)
    - $\tilde{C}t = tanh(W_C \cdot [h\{t-1\}, x_t] + b_C)$
4. **细胞状态更新**: C_t = f_t × $C\{t-1\} + i_t \times \tilde{C}t$
5. **输出门 (Output Gate)**: 决定输出哪些信息
    - o_t = σ(W_o·[h_{t-1}, x_t] + b_o)
    - h_t = o_t × tanh(C_t)

重要公式/概念:

- 遗忘门: f_t = σ(W_f·[h_{t-1}, x_t] + b_f)，决定遗忘多少旧信息
- 输入门: i_t = σ(W_i·[h_{t-1}, x_t] + b_i)，决定更新多少新信息
- 候选值: $\tilde{C}t = tanh(W_C \cdot [h\{t-1\}, x_t] + b_C)$
- 状态更新: C_t = f_t ⊙ $C\{t-1\} + i_t \odot \tilde{C}t$
- 输出门: o_t = σ(W_o·[h_{t-1}, x_t] + b_o)
- 隐状态: h_t = o_t ⊙ tanh(C_t)

关键要点:

- Cell State是关键，保存长期信息
- 三个门（遗忘、输入、输出）使用Sigmoid激活，输出0-1范围
- 有效缓解梯度消失问题
- 比标准RNN有更强的记忆能力
- 典型应用：机器翻译、语音识别、文本生成

概念对照与易混点

MMC vs SVC vs SVM

- **MMC（最大间隔分类器）**： 硬间隔，数据必须完全线性可分，不允许任何误分类
- **SVC（支持向量分类器）**：软间隔，允许部分误分类，通过参数C控制，适用于线性不可分数据
- **SVM（支持向量机）**：SVC + 核函数，可处理非线性问题，通过核技巧映射到高维空间

激活函数对比

- **Sigmoid**: 输出(0,1)，存在梯度消失，适合输出层（二分类）
- **Tanh**: 输出(-1,1)，零中心化，也存在梯度消失
- **ReLU**: 输出[0,+∞)，计算快，缓解梯度消失，但有Dead Neuron问题
- **Leaky ReLU**: 解决ReLU的Dead Neuron问题

FNN vs CNN vs RNN

- **FNN**: 全连接，适合表格数据，无特定结构假设
- **CNN**: 卷积+池化，适合图像数据，利用空间局部性
- **RNN/LSTM**: 循环连接，适合序列数据，捕捉时序依赖

梯度消失 vs 梯度爆炸

- **梯度消失**: 梯度接近0，浅层参数几乎不更新，网络难以学习

    - 原因: Sigmoid/Tanh梯度小、网络太深
    - 解决: ReLU、残差连接、合理初始化
- **梯度爆炸**: 梯度极大，参数更新剧烈，训练不稳定
    - 原因: 权重初始化过大、网络太深
    - 解决: 梯度裁剪、合理初始化、降低学习率

Pooling操作对比

- **Max Pooling**: 取区域最大值，保留最显著特征，常用
- **Average Pooling**: 取区域平均值，保留整体信息，平滑特征

核函数对比

- **线性核**: 等价于SVC，适合线性可分数据
- **多项式核**: 适合具有多项式关系的数据，参数d控制阶数
- **径向核(RBF)**: 最常用，可产生任意复杂边界，参数γ控制影响范围

关键参数总结

SVM相关

- **C (SVC和SVM)**: 正则化参数，控制误分类容忍度
    - C大: 间隔宽，容忍更多误分类，低方差高偏差
    - C小: 间隔窄，更严格，高方差低偏差
- **d (多项式核)**: 多项式阶数，控制决策边界复杂度
- **γ (径向核)**: 控制样本影响范围，γ大则影响范围小

神经网络相关

- **学习率α**: 梯度下降步长
    - α太大: 不收敛或震荡
    - α太小: 收敛太慢
- **隐藏层数和神经元数**: 控制模型复杂度
    - 更多层/神经元: 更强表达能力，但易过拟合
- **批大小 (Batch Size)**: 每次更新使用的样本数
- **Epoch数**: 完整遍历训练集的次数

CNN相关

- **Kernel Size**: 卷积核大小（如3×3, 5×5）
- **Stride**: 卷积核滑动步长
- **Padding**: 边缘填充大小
- **Pooling Size**: 池化窗口大小

训练技巧与注意事项

防止过拟合

1. 使用验证集(Validation Set)选择超参数

2. 正则化: L1/L2正则化
3. Dropout: 随机丢弃神经元
4. Early Stopping: 验证集性能不再提升时停止训练
5. 数据增强 (Data Augmentation): 增加训练样本多样性

提升训练效果

1. 合理初始化权重（如Xavier初始化、He初始化）
2. 批归一化 (Batch Normalization)
3. 使用合适的优化器（Adam、RMSprop等）
4. 学习率调度（学习率衰减）
5. 残差连接解决梯度消失

模型选择

- **表格数据**: FNN、SVM
- **图像数据**: CNN
- **序列数据**: RNN、LSTM
- **长序列**: LSTM、Transformer
- **小样本**: SVM（核方法）
- **大样本**: 深度神经网络

损失函数（补充）

分类损失

- **交叉熵损失 (Cross-Entropy)**: 用于分类问题，与Softmax配合
  - 二分类: $L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$
  - 多分类: $L = -\Sigma y_i \cdot \log(\hat{y}_i)$
- **Hinge Loss**: SVM使用的损失函数

回归损失

- **均方误差 (MSE)**: $L = (1/n)\Sigma(y_i - \hat{y}_i)^2$
- **平均绝对误差 (MAE)**: $L = (1/n)\Sigma|y_i - \hat{y}_i|$

快速查找表

方法	任务类型	优点	缺点	页码
MMC	二分类	简单、理论完善	仅适用于完全线性可分	p.20-30
SVC	二分类	允许误分类、更鲁棒	仅处理线性问题	p.31-44
SVM	二/多分类	处理非线性、泛化好	参数选择困难、计算复杂	p.45-52
FNN	分类/回归	通用、灵活	参数多、易过拟合	p.50-57
CNN	图像分类	参数少、利用空间结构	需要大量数据	p.30-51
RNN	序列建模	处理变长序列	梯度消失严重	p.53-66

LSTM	序列建模	学习长期依赖	计算复杂、参数多	p.68-74

考试建议：

1. 记住关键公式和约束条件
2. 理解各方法的适用场景和核心区别
3. 掌握超参数的作用和调节方向
4. 熟悉常见问题（梯度消失/爆炸、过拟合）的解决方案
5. 能够根据数据类型选择合适的模型

机器学习开卷考试复习资料 (Lecture 4-5)

快速索引

按任务分类

- **回归任务**: Polynomial Regression, Regression Splines, Local Regression, GAMs (Regression), Regression Tree
- **分类任务**: GAMs (Classification), Classification Tree
- **非线性建模**: Polynomial Regression, Step Functions, Regression Splines, Local Regression
- **集成学习**: Bagging, Boosting, Random Forest

按缩写字母排序

- **BS**: Basis Functions (基函数)
- **DT**: Decision Tree (决策树)
- **GAMs**: Generalized Additive Models (广义可加模型)
- **LR**: Local Regression (局部回归)
- **PR**: Polynomial Regression (多项式回归)
- **RBS**: Recursive Binary Splitting (递归二叉分裂)
- **RF**: Random Forest (随机森林)
- **RS**: Regression Splines (回归样条)
- **RSS**: Residual Sum of Squares (残差平方和)
- **SF**: Step Functions (阶梯函数)

详细方法总结

方法1：多项式回归 / Polynomial Regression

缩写： PR
页码： Lecture-4, p.6-11

解决问题：

- 任务类型：回归任务
- 典型输入：连续型预测变量 X
- 典型输出：连续型响应变量 Y

- 适用场景：预测变量与响应变量之间存在非线性关系时

工作原理：

1. 核心思想：用高次多项式替代线性模型,捕捉非线性关系
3. 优化目标：最小化残差平方和,使用最小二乘法估计系数
4. 训练流程：本质上是标准线性模型(预测变量为 $x_i, x_i^2, x_i^3, \ldots, x_i^d$),可直接用最小二乘回归求解

重要公式/概念：

- 多项式模型：$Y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i$
- 逻辑回归扩展(分类)：$P(y_i > 250|x_i) = \frac{\exp(\beta_0+\beta_1 x_i+\beta_2 x_i^2+\cdots+\beta_d x_i^d)}{1+\exp(\beta_0+\beta_1 x_i+\beta_2 x_i^2+\cdots+\beta_d x_i^d)}$

关键要点：

- 关键超参数：多项式阶数 d（通常不超过3或4,避免过拟合)
- 与线性回归的区别：允许拟合非线性曲线
- 注意事项：阶数过高会导致过拟合,边界区域可能产生不合理预测

方法2：阶梯函数 / Step Functions

缩写： SF

页码： Lecture-4, p.13-22

解决问题：

- 任务类型：回归/分类任务
- 典型输入：连续型预测变量 X
- 典型输出：分段常数预测
- 适用场景：需要将连续变量离散化,在不同区间使用不同常数预测

工作原理：

1. 核心思想：将预测变量 X 的取值范围划分为 K+1 个不重叠区间,在每个区间内使用常数预测
2. 关键步骤：创建截断点 $c_1, c_2, \ldots, c_K$,定义示性函数 $C_j(X) = I(c_j < X < c_{j+1})$
3. 优化目标：使用最小二乘法拟合分段常数模型
4. 模型形式：$Y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i$

重要公式/概念：

- 示性函数：$I(\cdot)$ 当条件成立时返回1,否则返回0
- 分段常数：对于任意 X 值,至多只有一个 $C_j(X)$ 非零
- 预测值：当 $c_j \leq X < c_{j+1}$ 时,预测值为 $\beta_0 + \beta_j$
- 逻辑回归扩展：$P(y_i > 250|x_i) = \frac{\exp(\beta_0+\beta_1 C_1(x_i)+\cdots+\beta_K C_K(x_i))}{1+\exp(\beta_0+\beta_1 C_1(x_i)+\cdots+\beta_K C_K(x_i))}$

关键要点：

- 关键超参数：截断点位置和数量 K
- 与多项式回归的区别：产生分段常数而非连续曲线
- 注意事项：可能丢失变量的有序信息,拟合不够光滑

方法3：基函数 / Basis Functions

缩写： BS

页码： Lecture-4, p.23

解决问题：

- 任务类型：回归任务的通用框架
- 典型输入：原始预测变量 X
- 典型输出：通过基函数变换后的线性组合
- 适用场景：需要灵活建模非线性关系的各种场景

工作原理：

1. 核心思想：定义一组基函数 $b_1(X), b_2(X), \ldots, b_K(X)$,将原始变量 X 变换后进行线性建模
2. 关键步骤：选定基函数形式,拟合线性模型 $Y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i$
3. 统一框架：多项式回归和阶梯函数都是基函数方法的特例
4. 优化目标：使用最小二乘法估计系数

重要公式/概念：

- 基函数模型：$Y_i = \beta_0 + \sum_{j=1}^{K} \beta_j b_j(x_i) + \epsilon_i$
- 多项式基：$b_j(x_i) = x_i^j$
- 阶梯函数基：$b_j(x_i) = I(c_j \leq x_i < c_{j+1})$

关键要点：

- 关键超参数：基函数的形式和数量
- 优势：提供统一的建模框架,可灵活选择基函数
- 注意事项：基函数形式需要在建模前确定

方法4：回归样条 / Regression Splines

缩写： RS

页码： Lecture-4, p.25-36

解决问题：

- 任务类型：回归任务
- 典型输入：连续型预测变量 X
- 典型输出：连续且光滑的分段多项式预测
- 适用场景：需要灵活拟合非线性关系,同时保证连续性和光滑性

工作原理：

1. 核心思想：在不同区域拟合低次多项式,在结点处保证连续性和光滑性约束
2. 关键步骤：选择结点位置 $\xi_1, \xi_2, \ldots, \xi_K$,在每个区域拟合三次多项式,添加连续性和导数连续性约束
3. 三次样条：要求函数及其一阶、二阶导数在结点处连续
4. 训练流程：使用截断幂基函数表示,通过最小二乘法拟合

重要公式/概念：

- 三次样条基表示：$Y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i$
- 截断幂基函数：$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$
- 自由度：K 个结点的三次样条有 K+4 个自由度
- 自然三次样条：在边界外强制为线性,减少边界区域的方差

关键要点：

- 关键超参数：结点数量 K 和位置
- 与多项式回归的区别：分段拟合更稳定,边界行为更好
- 优势：可在函数变化快的区域放置更多结点,变化慢的区域放置较少结点
- 注意事项：结点选择影响拟合质量,可通过交叉验证选择

方法5：局部回归 / Local Regression

缩写： LR
页码： Lecture-4, p.38-40

解决问题：

- 任务类型：回归任务
- 典型输入：连续型预测变量 X
- 典型输出：目标点 x_0 处的预测值
- 适用场景：不需要全局模型,只需要局部拟合的场景

工作原理：

1. 核心思想：对于每个目标点,仅使用其邻近的训练样本进行加权拟合
2. 关键步骤：
    - 选择目标点 x_0 的 k 个最近邻
    - 为每个邻居分配权重(距离越近权重越大)
    - 使用加权最小二乘法拟合局部模型
    - 用局部模型在 x_0 处预测
3. 优化目标：最小化加权残差平方和 $\sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2$
4. 非参数方法：不需要预先指定全局函数形式

重要公式/概念：

- 加权最小二乘：$\min_{\beta_0,\beta_1} \sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2$
- 权重函数：$K_{i0} = K(x_i, x_0)$,最远的邻居权重为0,最近的权重最大
- 预测值：$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$

关键要点：

- 关键超参数：邻域比例 s = k/n (控制光滑度)
- 优势：非常灵活,能自动适应局部数据特征
- 注意事项：计算成本较高(每个预测点需要重新拟合),s 过小会过拟合,过大会过度平滑

方法6：广义可加模型 / Generalized Additive Models

缩写： GAMs
页码： Lecture-4, p.42-52

解决问题：

- 任务类型：回归和分类任务
- 典型输入：多个预测变量 $X_1, X_2, \ldots, X_p$
- 典型输出：响应变量 Y 的预测

- 适用场景：多变量非线性建模,同时保持模型可解释性

工作原理：

1. 核心思想：对每个预测变量使用非线性函数,保持模型的可加性
2. 关键步骤：为每个变量 X_j 独立拟合非线性函数 f_j,将所有贡献相加
3. 回归模型：$Y_i = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i$
4. 分类模型(逻辑回归)：$\log \frac{p(X)}{1-p(X)} = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$

重要公式/概念：

- GAM回归：$Y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i$
- GAM分类：$\log \frac{p(X)}{1-p(X)} = \beta_0 + \sum_{j=1}^{p} f_j(X_j)$
- 构建块：f_j 可以是自然样条、局部回归、多项式等任何非线性函数
- 分类变量：使用阶梯函数(每个水平一个常数)

关键要点：

- 关键超参数：每个 f_j 的自由度(控制光滑度)
- 优势：可同时处理定量和定性变量,模型可解释性强,可单独分析每个变量的影响
- 与线性模型的区别：允许非线性关系,预测更准确
- 局限性：仅能捕捉可加效应,可能错过重要交互作用(可手动添加交互项 $\times X_k$)

方法7：决策树 / Decision Tree

缩写： DT

页码： Lecture-5, p.8-12

解决问题：

- 任务类型：分类和回归任务
- 典型输入：任意类型的特征(连续、离散)
- 典型输出：类别标签(分类)或数值(回归)
- 适用场景：需要可解释性强的模型,特征间存在复杂非线性关系

工作原理：

1. 核心思想：通过分层和分割将特征空间划分为若干简单区域,每个区域对应一个预测值
2. 树结构组成：
    - 根节点(root node)：包含所有数据
    - 非叶子节点(non-leaf node)：表示测试条件
    - 分支(branches)：连接节点
    - 叶节点(leaf node)：包含预测结果
3. 预测方式：对于待预测样本,按照树的分裂规则找到其所属叶节点,使用该叶节点的训练样本响应值的均值(回归)或众数(分类)作为预测

重要公式/概念：

- 区域划分：将特征空间划分为 $R_1, R_2, \ldots, R_J$ 个不重叠区域
- 回归预测：$\hat{y}_{R_j}$ 为区域 R_j 中训练样本的 y 均值
- 分类预测：选择区域 R_j 中最常见的类别

关键要点：

- 优势：易于理解和解释,有直观的图形表示,可处理非线性关系
- 与线性模型的区别：非参数方法,不假设数据分布
- 注意事项：单棵树容易过拟合,预测准确性通常不如其他方法

方法8：回归树 / Regression Tree

缩写： RT
页码： Lecture-5, p.14-41

解决问题：

- 任务类型：回归任务
- 典型输入：连续或离散特征
- 典型输出：连续数值
- 适用场景：需要预测连续响应变量,且需要模型可解释性

工作原理：

1. 核心思想：递归地将特征空间二分,使每个区域内的 RSS 最小
2. 关键步骤：
    - **Step 1**: 使用递归二叉分裂划分特征空间
    - **Step 2**: 对每个区域,用训练样本的 y 均值作为预测
3. 递归二叉分裂(Recursive Binary Splitting)：
    - 自顶向下贪心算法
    - 对每个特征 X_j 和截断点 s,考虑划分 $R_1(j,s) = \{X|X_j < s\}$ 和 $R_2(j,s) = \{X|X_j \geq s\}$
    - 选择使 RSS 最小的 j 和 s
4. 树剪枝：生成大树后进行剪枝以避免过拟合

重要公式/概念：

- 损失函数(RSS)：$\sum_{j=1}^{J}\sum_{(x_{i1},x_{i2})\in R_j}(y_i - \hat{y}_{R_j})^2$
- 分裂准则：$\min_{j,s}\left[\sum_{i:x_i\in R_1(j,s)}(y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i\in R_2(j,s)}(y_i - \hat{y}_{R_2})^2\right]$
- 预测值：$\hat{y}_{R_j}$ 为区域 R_j 内所有训练样本 y 的均值

关键要点：

- 关键超参数：最大深度、最小叶节点样本数、剪枝参数 α
- 优势：可解释性强,自动进行特征选择
- 注意事项：容易过拟合,需要剪枝控制复杂度

方法9：树剪枝 / Tree Pruning (代价复杂性剪枝)

缩写： Cost Complexity Pruning, CCP
页码： Lecture-5, p.36-41

解决问题：

- 任务类型：决策树模型优化
- 目的：防止过拟合,选择最优子树
- 适用场景：决策树模型复杂度控制

工作原理：

1. 核心思想：在大树中选择一个子树,平衡拟合误差和模型复杂度
2. 关键步骤：
    - 生长一棵很大的树 T_0
    - 对于给定的 α,选择最小化代价复杂性准则的子树
    - 使用 K 折交叉验证选择最优 α
    - 返回对应的最优子树
3. 优化目标：最小化 $\alpha|T| + \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2$

重要公式/概念：

- 代价复杂性准则： $\alpha|T| + \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2$
- $|T|$：子树 T 的叶节点数量(模型复杂度)
- α：调节参数,控制复杂度与拟合的权衡
- 当 $\alpha = 0$ 时,子树就是完整的大树 T_0
- 当 α 增大时,选择的子树会更小

关键要点：

- 关键超参数：α (通过交叉验证选择)
- 选择方法：K 折交叉验证,计算每个 α 的平均测试误差
- 与完整树的区别：剪枝后的树更简单,泛化能力更强
- 注意事项：α 过大会导致树过于简单(欠拟合)

方法10：分类树 / Classification Tree

缩写： CT
页码： Lecture-5, p.48-58

解决问题：

- 任务类型：分类任务
- 典型输入：连续或离散特征
- 典型输出：类别标签
- 适用场景：需要预测定性响应变量

工作原理：

1. 核心思想：与回归树类似,但使用不同的分裂准则和预测方式
2. 预测方式：使用区域内最常见的类别(众数)
3. 分裂准则：不能使用 RSS,改用以下指标：
    - 分类错误率
    - 基尼指数(Gini Index)
    - 熵(Entropy)
4. 通常使用基尼指数或熵来生长树,使用分类错误率来剪枝

重要公式/概念：

- 分类错误率： $E = 1 - \max_k(\hat{p}_{mk})$
- 基尼指数： $G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$
- 熵： $H = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$

- $\hat{p}_{mk}$：第 m 个区域中第 k 类所占的比例
- 节点纯度：基尼指数或熵较小表示节点纯度高(样本主要来自同一类)

关键要点：

- 关键超参数：最大深度、最小叶节点样本数、分裂准则选择
- 分裂准则选择：基尼指数和熵对节点纯度更敏感,适合树生长;分类错误率适合剪枝
- 与回归树的区别：使用众数而非均值预测,使用不同的分裂准则
- 注意事项：基尼指数和熵在数值上很相似

方法11：集成学习 - Bagging (Bootstrap Aggregation)

缩写： Bagging, 装袋法
页码： Lecture-5, p.82

解决问题：

- 任务类型：减少预测方差,提高模型稳定性
- 典型输入：训练数据集
- 典型输出：多个模型的聚合预测
- 适用场景：单个模型方差较大,容易过拟合

工作原理：

1. 核心思想：通过自助法(bootstrap)生成多个训练集,训练多个独立模型,对预测结果取平均(回归)或投票(分类)
2. 关键步骤：
    - 从训练数据中有放回地抽取 B 个 bootstrap 样本
    - 对每个样本独立训练一个模型
    - 回归任务：对 B 个模型的预测取平均
    - 分类任务：对 B 个模型的预测进行投票
3. 优化目标：减少模型方差,不改变偏差

重要公式/概念：

- Bootstrap 抽样：从原始数据集有放回地抽取 n 个样本
- 回归预测：$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{b}(x)$
- 分类预测：多数投票
- 袋外误差(OOB Error)：对于每个样本,使用未包含该样本的模型进行预测,估计测试误差

关键要点：

- 关键超参数：模型数量 B (通常越大越好,但计算成本增加)
- 优势：显著减少方差,提高稳定性,可并行训练
- 与单个模型的区别：牺牲一定可解释性换取更高的预测准确性
- 注意事项：对低方差模型(如线性回归)效果不明显,主要用于高方差模型(如决策树)

方法12：集成学习 - Boosting

缩写： Boosting, 提升法
页码： Lecture-5, p.81, 83-84

解决问题：

- 任务类型：顺序集成学习,提高模型准确性
- 典型输入：训练数据集
- 典型输出：加权模型的组合预测
- 适用场景：需要逐步改进模型,重点关注之前模型预测不好的样本

工作原理：

1. 核心思想：序贯地训练多个弱学习器,每个新模型关注前面模型的残差(预测误差)
2. 关键步骤：
    - 初始化 $f(x) = 0$ 和残差 $r_i = y_i$
    - 对 b = 1 到 B：
        - 用当前残差拟合一个浅树 f^b
        - 更新模型：$f(x) \leftarrow f(x) + \lambda f^b(x)$
        - 更新残差：$r_i \leftarrow r_i - \lambda f^b(x_i)$
    - 最终模型：$f(x) = \sum_{b=1}^{B} \lambda f^b(x)$
3. 慢学习：使用收缩参数 λ 控制每棵树的贡献

重要公式/概念：

- 更新规则：$f(x) \leftarrow f(x) + \lambda f^b(x)$
- 残差更新：$r_i \leftarrow r_i - \lambda f^b(x_i)$
- 最终模型：$f(x) = \sum_{b=1}^{B} \lambda f^b(x)$
- 收缩参数：λ (通常为小值如 0.01 或 0.001)

关键要点：

- 关键超参数：树的数量 B,收缩参数 λ,每棵树的深度 d
- 与 Bagging 的区别：Boosting 树序贯生长(依赖前面的树),Bagging 树独立生长
- 优势：通常比 Bagging 准确性更高,可使用浅树(高偏差低方差)
- 注意事项：对噪声和异常值更敏感,容易过拟合(需要调节 B 和 λ)

Random Forest 随机森林

Random forests provide an improvement by way of a random small tweak that decorrelates the trees. 随机森林通过对树做去相关处理，实现了树的提升。

Random Forest 随机森林

随机森林通过对树做去相关处理，实现了对装袋法树的提升。 在建立决策树时，每考虑树上的一个分裂点，都要从全部p个预 测变量中选出一个包含m个预测变量的随机样本作为候选变量。 这个分裂点所用的预测变量只能从m个变量中选择 。每个分裂点处都进行重新抽样随机森林通过对树做去相关处理，实现了对树的提升。 随机森林集成多个决策树，每个决策树基于自助法采样数据集构建，分裂点随机选择特征，最后投票（分类）或平均（回归）整合预测，提升准确性和泛化力。

- **去相关**

随机森林通过强制每个分裂只考虑预测变量的一个子集来克服这个问题

- **装袋法和随机森林之间的区别**

预测变量子集大小 m 的选择。例如，如果使用 $m = p$ 构建随机森林，那么这实际上就等于装袋法。

使用 $m = \sqrt{p}$ 的随机森林在测试误差和 OOB 误差方面都优于装袋法。

集成学习

- **总结**
    - 在装袋法中，树是在观测的随机样本上独立生长的。因此，这些树往往彼此非常相似。因此，装袋法可能会陷入局部最优，并且可能无法彻底探索模型空间。
    - 在随机森林中，树再次是在观测的随机样本上独立生长的。然而，每棵树上的每次分裂都是使用特征的随机子集进行的，从而使树之间去相关，并相对于装袋法更彻底地探索模型空间。
    - 在提升法中，我们只使用原始数据，不抽取任何随机样本。树是连续生长的，使用“慢速”学习方法：每棵新

树都拟合之前树留下的残差信号，并在使用前进行收缩。

方法13：递归二叉分裂 / Recursive Binary Splitting

缩写： RBS

页码： Lecture-5, p.28-31

解决问题：

- 任务类型：决策树构建算法
- 目的：确定树的分裂变量和分裂点
- 适用场景：构建回归树或分类树的内部节点

工作原理：

1. 核心思想：自顶向下的贪心算法,每次选择使损失函数下降最多的分裂
2. 关键步骤：
    - **自顶向下(Top-down)**：从根节点开始,所有样本在一个区域
    - **贪心(Greedy)**：每步选择当前最优分裂,不考虑未来
    - 对每个特征 X_j 和截断点 s,考虑所有可能的划分
    - 选择使损失函数(RSS或基尼指数)最小的 (j, s)
    - 递归地对子区域重复该过程
3. 停止准则：达到最小叶节点样本数或最大深度

重要公式/概念：

- 区域划分：$R_1(j,s) = \{X|X_j < s\}$ 和 $R_2(j,s) = \{X|X_j \geq s\}$
- 回归树分裂准则：$\min_{j,s}\left[\sum_{i:x_i\in R_1(j,s)}(y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i\in R_2(j,s)}(y_i - \hat{y}_{R_2})^2\right]$
- 分类树分裂准则：最小化基尼指数或熵

关键要点：

- 关键特点：贪心算法,计算效率高但可能不是全局最优
- 优势：易于实现,适用于大数据集
- 局限性：可能导致过拟合(生长过大的树),需要后续剪枝

概念对照与易混点

Polynomial Regression vs Step Functions

- **核心区别**：多项式回归产生连续光滑曲线,阶梯函数产生分段常数
- **适用场景**：多项式适合整体趋势建模,阶梯函数适合离散化特征

Regression Splines vs Polynomial Regression

- **核心区别**：样条是分段多项式(更灵活),多项式回归是全局多项式
- **边界行为**：样条(尤其自然三次样条)在边界表现更稳定,多项式容易在边界产生不合理波动

Local Regression vs Regression Splines

- **核心区别**：局部回归不需要预先指定全局结构,样条需要选择结点
- **计算成本**：局部回归每个预测点都需要重新拟合,计算成本更高

Bagging vs Boosting

- **训练方式**：Bagging 并行训练独立模型,Boosting 序贯训练依赖模型
- **目标**：Bagging 减少方差,Boosting 同时减少偏差和方差
- **鲁棒性**：Bagging 对噪声更鲁棒,Boosting 对噪声更敏感

Regression Tree vs Classification Tree

- **预测方式**：回归树用均值预测,分类树用众数预测
- **分裂准则**：回归树用 RSS,分类树用基尼指数/熵/分类错误率
- **输出类型**：回归树输出连续值,分类树输出类别标签

Gini Index vs Entropy

- **数值表现**：两者在数值上很相似
- **使用场景**：都用于衡量节点纯度,值越小节点越纯
- **计算效率**：基尼指数计算稍快(不需要对数运算)

GAMs vs Standard Linear Model

- **非线性**：GAMs 允许每个变量的非线性函数,线性模型只允许线性关系
- **可加性**：两者都是可加模型,便于解释
- **灵活性**：GAMs 更灵活但仍保持可解释性

Natural Cubic Spline vs Standard Cubic Spline

- **边界约束**：自然三次样条在边界外强制为线性,标准三次样条无此约束
- **自由度**：自然三次样条自由度更少(更稳定)
- **边界表现**：自然三次样条在边界区域方差更小,预测更稳定

完整算法流程总结

回归树构建算法(含剪枝)

1. 使用递归二叉分裂生长大树,停止条件为叶节点样本数少于阈值
2. 对大树应用代价复杂性剪枝,生成一系列候选子树(对应不同 α)
3. 使用 K 折交叉验证选择最优 α:
    - 对每个折,在训练集上重复步骤1和2
    - 在验证集上评估每个 α 对应子树的 MSE
    - 对每个 α,计算 K 个折的平均 MSE
4. 选择平均 MSE 最小的 α,返回对应的子树

局部回归算法

1. 收集距离 x_0 最近的 k = s×n 个训练点
2. 为每个邻居分配权重 K_{i0} ,最远的权重为0,最近的权重最大
3. 使用加权最小二乘拟合：$\min_{\beta_0,\beta_1} \sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2$
4. 预测值：$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$

Boosting 算法(回归树)

1. 初始化 $f(x) = 0$ 和残差 $r_i = y_i$ (对所有训练样本)
2. 对 b = 1, 2, ..., B 重复:
    - 在数据 (X, r) 上拟合深度为 d 的树 f^b
    - 更新模型: $f(x) \leftarrow f(x) + \lambda f^b(x)$
    - 更新残差: $r_i \leftarrow r_i - \lambda f^b(x_i)$
3. 输出最终模型: $f(x) = \sum_{b=1}^{B} \lambda f^b(x)$

关键术语中英对照

- **Bias-Variance Tradeoff** - 偏差-方差权衡
- **Bootstrap** - 自助法
- **Cross-Validation** - 交叉验证
- **Cutpoint** - 截断点
- **Degrees of Freedom** - 自由度
- **Ensemble Learning** - 集成学习
- **Knot** - 结点
- **Leaf Node** - 叶节点
- **Nonparametric** - 非参数
- **Overfitting** - 过拟合
- **Piecewise** - 分段
- **Predictor Space** - 预测变量空间
- **Pruning** - 剪枝
- **Residual** - 残差
- **Root Node** - 根节点
- **Smoothing** - 平滑
- **Spline** - 样条
- **Stratification** - 分层
- **Terminal Node** - 终端节点(叶节点)
- **Truncated Power Basis** - 截断幂基

考试重点提示

高频考点

1. **多项式回归**: 阶数选择原则,过拟合风险
2. **回归样条**: 结点概念,自由度计算,自然三次样条的边界约束
3. **决策树**: 递归二叉分裂算法,剪枝必要性
4. **分类树**: 基尼指数和熵的计算与含义
5. **Bagging vs Boosting**: 核心区别,适用场景
6. **GAMs**: 可加性的含义,优势与局限

公式重点

- 多项式模型：$Y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i$
- 阶梯函数：$Y_i = \beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i$
- 截断幂基：$h(x, \xi) = (x - \xi)_+^3$
- 基尼指数：$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$
- 熵：$H = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$
- 代价复杂性：$\alpha|T| + \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2$
- GAM 回归：$Y_i = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i$

概念辨析重点

- 全局方法 vs 局部方法
- 参数方法 vs 非参数方法
- 连续性 vs 光滑性
- 偏差 vs 方差
- 自顶向下 vs 自底向上
- 贪心算法的优缺点

复习建议：

1. 重点理解每种方法的核心思想和适用场景
2. 熟记关键公式,理解其物理含义
3. 掌握易混概念的核心区别
4. 练习手工计算简单示例(如基尼指数、树的分裂)
5. 理解超参数对模型的影响

ML-Lecture-3 Classification - 开卷考试复习资料

快速索引

按任务分类

- **分类任务**: Logistic Regression, Naive Bayes, Linear Discriminant Analysis (LDA), k-Nearest Neighbor (kNN)
- **回归任务**: Linear Regression, k-Nearest Neighbor (kNN)
- **多分类策略**: OvO, OvR, MvM/ECOC

按缩写对照表

- **ECOC**: Error Correcting Output Codes (纠错输出码)
- **FN**: False Negative (假负例)
- **FP**: False Positive (假正例)
- **FPR**: False Positive Rate (假正例率)
- **kNN**: k-Nearest Neighbor (k近邻)
- **LDA**: Linear Discriminant Analysis (线性判别分析)
- **LR**: Linear Regression (线性回归) / Logistic Regression (逻辑回归)

- **MLE**: Maximum Likelihood Estimation (最大似然估计)
- **MvM**: Many vs Many (多对多)
- **OvO**: One vs One (一对一)
- **OvR**: One vs Rest (一对其余)
- **ROC**: Receiver Operating Characteristic (受试者工作特征曲线)
- **TN**: True Negative (真负例)
- **TP**: True Positive (真正例)
- **TPR**: True Positive Rate (真正例率)

详细方法总结

方法1: 线性回归 / Linear Regression

缩写: LR
页码: Lecture 3-1 p.2

解决问题:

- 任务类型: 回归任务
- 典型输入: 一个或多个连续/离散特征 X
- 典型输出: 连续值 Y
- 适用场景: 预测连续数值型目标变量

工作原理:

1. 核心思想: 通过拟合线性函数建立输入与输出之间的关系
2. 一元线性回归: $Y \approx \beta_0 + \beta_1 X$
3. 多元线性回归: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p + \varepsilon$
4. 优化目标: 最小化预测值与真实值之间的平方误差

重要公式/概念:

- 损失函数: $J(\beta_0, \beta_1) = \Sigma^n_{i=1}(y_i - (\beta_0 + \beta_1 x_i))^2$
- 估计方法: 最小二乘估计 (Least Square Estimation) 或梯度下降 (Gradient Descent)

关键要点:

- 不适合直接用于分类任务
- 原因1: 输出值域为(-∞, +∞),但分类需要{0,1}
- 原因2: 决策边界对异常值敏感
- 原因3: 多分类时类别编码会影响预测结果

方法2: k近邻 / k-Nearest Neighbor

缩写: kNN
页码: Lecture 3-1 p.3

解决问题:

- 任务类型: 分类或回归
- 典型输入: 特征向量
- 典型输出: 类别标签(分类)或数值(回归)

- 适用场景: 简单的监督学习任务,无需训练过程

工作原理:

1. 核心思想: 基于距离度量找到最近的k个训练样本
2. 分类任务: 对新样本,根据其k个最近邻训练样本的类别,通过多数表决进行预测
3. 回归任务: 对新样本,根据其k个最近邻训练样本标签值的均值作为预测值
4. 距离度量: 常用欧氏距离、曼哈顿距离等

重要公式/概念:

- 欧氏距离: $d(x,y) = \sqrt{(\Sigma(x_i-y_i)^2)}$
- 预测规则(分类): $\hat{y} = mode\{y^{n^1}, y^{n^2}, ..., y^{nk}\}$
- 预测规则(回归): $\hat{y} = mean\{y^{n^1}, y^{n^2}, ..., y^{nk}\}$

关键要点:

- 超参数k的选择很重要
- 计算成本高,需要计算与所有训练样本的距离
- 对特征尺度敏感,通常需要标准化
- 属于懒惰学习(lazy learning),无显式训练过程

方法3: 逻辑回归 / Logistic Regression

缩写: LR, Logit

页码: Lecture 3-1 p.23-40

解决问题:

- 任务类型: 二分类或多分类
- 典型输入: 连续或离散特征向量
- 典型输出: 类别概率 P(Y=1|X)
- 适用场景: 线性可分或近似线性可分的分类问题

工作原理:

1. 核心思想: 使用Sigmoid函数将线性组合映射到(0,1)区间,表示概率
2. 二分类模型: 在二分类中，Y 只能是 0 或 1。 $P(Y=1) = 1/(1 + e^{-(\beta_0+\beta_1X)})$
3. 多元逻辑回归: $P(Y=1) = e^{(\beta_0+\beta_1X_1+...+\beta_pX_p)}/(1 + e^{(\beta_0+\beta_1X_1+...+\beta_pX_p)})$
4. 训练方法: 最大似然估计(MLE)
5. 决策规则: 通常以0.5为阈值,P(Y=1)>0.5则预测为1

重要公式/概念:

- Sigmoid函数: $g(z) = 1/(1 + e^{-z})$
- 几率(odds): $P(Y=1)/(1-P(Y=1)) = e^{(\beta_0+\beta_1X)}$
- 对数几率(logit): $\log(P(Y=1)/(1-P(Y=1))) = \beta_0 + \beta_1X$
- 估计系数 - 最大似然估计 MLE目标: MLE 是一种在给定一些观测数据的情况下，估计假设概率分布的参数的方法。假设数据样本是 $\{(x_i, y_i)\}_{i=1}^n$，其中 $x_i = [x_{i1}, \ldots, x_{ip}]$ 是预测变量的初始实现（样本）。MLE 通过求解以下问题来找到 $\{\beta_i\}_{i=0}^p$，$\max \Pi^{n}_{i=1}P(Y=y_i) = \max \Sigma^{n}_{i=1}\log P(Y=y_i)$
- 多项逻辑回归(Multinomial): $P(Y=k|X) = e^{(\beta_{k0}+\beta_{k1}X_1+...+\beta_{kp}X_p)} / \Sigma^{K}_{i=1}e^{(\beta_{l0}+\beta_{l1}X_1+...+\beta_{lp}X_p)}$
- *将二分类逻辑回归方法扩展到 $K > 2$ 个类别的设置。*
- ***选择一个单独的类作为基线（基类），例如，选择第 K^{th} 类作为基线。***

- *对于每个* $k = 1, 2, \ldots, K-1$

$$log(\frac{P(Y=k \mid X=x)}{P(Y=K \mid X=x)}) = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p$$

$$P(Y=k \mid X=x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}$$

$$P(Y=K \mid X=x) = \frac{1}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}$$

关键要点:

- Sigmoid函数连续可导,可以使用梯度下降优化
- 输出可解释为概率
- 对于多分类可使用Multinomial Logistic Regression或Softmax
- 相比线性回归,输出限制在[0,1]区间内
- 决策边界是线性的

方法4: 朴素贝叶斯 / Naive Bayes

缩写: NB

页码: Lecture 3-2 p.5-18

解决问题:

- 任务类型: 分类(二分类或多分类)
- 典型输入: 特征向量(离散或连续)
- 典型输出: 类别标志

适用场景: 文本分类、垃圾邮件检测等特征相对独立的场景

工作原理:

1. 核心思想: 基于贝叶斯定理和条件独立性假设进行分类
2. 贝叶斯定理: P(类别|特征) = P(特征|类别)·P(类别) / P(特征)
3. 条件独立性假设: 在给定类别Y的条件下,各个特征相互独立
   $P(X_1,X_2,...,X_n|Y) = \Pi^n_{j=1}P(X_j|Y)$
4. 决策规则: 选择后验概率最大的类别
   $\hat{y} = \text{argmax}_k\ P(Y=k|X) = \text{argmax}_k\ P(Y=k)\cdot\Pi_j\ P(X_j|Y=k)$

重要公式/概念:

- 贝叶斯定理: P(A|B) = P(B|A)·P(A) / P(B)
- 后验概率: P(类别|特征)
- 先验概率: P(类别) - 可通过训练集中各类的占比估计,$\pi_k = n_k/n$
- 似然: P(特征|类别)
- 条件独立: $P(X_1,...,X_p|Y=k) = f_{k1}(X_1) \times f_{k2}(X_2) \times ... \times f_{kp}(X_p)$
- 全概率公式: $P(A) = \Sigma^n_{i=1}P(B_i)P(A|B_i)$
- 最终公式: $P(Y=k|X) = \pi_k \times f_{k1}(X_1) \times ... \times f_{kp}(X_p) / \Sigma^K_{j=1}\pi_j f_{j1}(X_1) \times ... \times f_{jp}(X_p)$

关键要点:

- "朴素"指的是条件独立性假设,这在现实中往往不成立
- 对于定性特征,$f_{kj}(x_j)$通过统计第j个特征在第k类中的占比得到
- 对于定量特征,通常假设服从某种分布(如正态分布)
- 计算简单高效,适合高维数据
- 对特征独立性假设敏感

方法5: 线性判别分析 / Linear Discriminant Analysis

缩写: LDA

页码: Lecture 3-2 p.19-36

解决问题:

- 任务类型: 分类(二分类或多分类)和降维
- 典型输入: 连续特征向量
- 典型输出: 类别标签或降维后的特征
- 适用场景: 线性可分问题,同时需要降维和分类

工作原理:

1. 核心思想: 通过数据降维将多类别分离,找到最佳投影方向
2. 几何视角: 找到投影方向ω使得
   - 不同类的中心点尽量远(类间差异大)
   - 同类的点尽量紧凑(类内差异小)
3. 概率视角(p=1): 假设每类X服从正态分布 $X|Y=k \sim N(\mu_k, \sigma^2)$,使用贝叶斯定理
4. 优化目标: max J = ω^T S_β ω / ω^T S_v ω

重要公式/概念:

- 类间散度矩阵: $S_\beta = (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T$
- 类内散度矩阵: $S_v = \Sigma_0 + \Sigma_1$
- 优化目标: $J = ||\omega^T \mu_0 - \omega^T \mu_1||^2 / (\omega^T \Sigma_0 \omega + \omega^T \Sigma_1 \omega)$
- 约束优化: $\min(-\omega^T S_\beta \omega)$ s.t. $\omega^T S_v \omega = 1$
- 解: $S_\beta \omega = \lambda S_v \omega$ (广义特征值问题)
- 概率判别: $P(Y=k|X=x) = \pi_k f_k(x) / \Sigma^K_{l=1} \pi_l f_l(x)$
- 判别函数(p=1): $\delta_k(x) = x \cdot \mu_k/\sigma^2 - \mu_k^2/(2\sigma^2) + \log(\pi_k)$
- 多元正态分布: $f(x) = 1/((2\pi)^{(p/2)}|\Sigma|^{(1/2)}) \exp(-1/2(x-\mu)^T \Sigma^{(-1)}(x-\mu))$
- 决策边界(K=2): $x = (\mu_1 + \mu_2)/2$ (当$\pi_1=\pi_2$时)

关键要点:

- 同时实现降维和分类
- 参数估计: $\mu_k = 1/n_k \Sigma_{i:y_i=k} x_i$, $\sigma^2 = 1/n \Sigma^K_{k=1} \Sigma_{i:y_i=k}(x_{ik}-\mu_k)^2$
- 假设各类协方差相同(共享协方差矩阵)
- LDA降维最多到K-1维(K为类别数)
- 决策边界是线性的
- 对异常值敏感

方法6: 多分类拆分策略 - 一对一 / One vs One

缩写: OvO

页码: Lecture 3-1 p.11-14

解决问题:

- 任务类型: 将多分类问题拆分为多个二分类问题
- 适用场景: 当已有二分类器时,通过组合解决多分类问题

工作原理:

1. 核心思想: 将N个类别两两配对,构建二分类任务
2. 训练过程: 为每一对类别(C_i, C_j)训练一个二分类器,共产生N(N-1)/2个分类器
3. 测试过程: 对新样本,所有分类器进行预测,通过投票得到最终类别
4. 预测规则: 选择得票最多的类别

重要公式/概念:

- 分类器数量: N(N-1)/2 (N为类别数)
- 投票机制: 每个分类器投票给其预测的类别,最终选择票数最多的类别

关键要点:

- 训练集较小,每个分类器只使用两个类别的数据
- 分类器数量较多,存储和预测开销较大
- 对不平衡数据相对鲁棒

方法7: 多分类拆分策略 - 一对其余 / One vs Rest

缩写: OvR, OvA (One vs All)

页码: Lecture 3-1 p.15-16

解决问题:

- 任务类型: 将多分类问题拆分为多个二分类问题
- 适用场景: 当已有二分类器时,通过组合解决多分类问题

工作原理:

1. 核心思想: 每次将一个类作为正例,所有其他类作为反例
2. 训练过程: 为每个类别训练一个二分类器,共产生N个分类器
3. 测试过程: 对新样本,所有分类器输出预测概率或置信度
4. 预测规则: 选择预测概率/置信度最高的类别

重要公式/概念:

- 分类器数量: N (N为类别数)
- 决策函数: $\hat{y} = \text{argmax}_k\ f_k(x)$, 其中$f_k(x)$是第k个分类器的输出

关键要点:

- 分类器数量少于OvO
- 每个分类器的训练集较大(使用所有数据)
- 可能存在类别不平衡问题
- 计算效率较高

方法8: 多分类拆分策略 - 多对多/纠错输出码 / Many vs Many / ECOC

缩写: MvM, ECOC
页码: Lecture 3-1 p.18-21

解决问题:

- 任务类型: 将多分类问题拆分为多个二分类问题
- 适用场景: 需要更强纠错能力的多分类任务

工作原理:

1. 核心思想: 每次将若干类作为正例,若干其他类作为反例
2. 编码阶段: 对N个类别做M次划分,每次划分产生一个二分类训练集,训练M个分类器
3. 编码矩阵: 每行代表一个类别,每列代表一个分类器
    - 二元码: +1(正类), -1(反类)
    - 三元码: +1(正类), -1(反类), 0(停用类)
4. 解码阶段: M个分类器对测试样本预测,组成预测编码,与各类别编码比较,返回距离最小的类别

重要公式/概念:

- 编码矩阵: N×M矩阵,N为类别数,M为分类器数量
- 海明距离: 两个编码对应位上不同的位数
- 码距: 编码矩阵中任意两个类别编码的最小海明距离
- 决策规则: $\hat{y} = \text{argmin}_k\ d(c_k, \hat{c})$, 其中$c_k$是类别k的编码,$\hat{c}$是预测编码,d是距离函数

关键要点:

- OvO和OvR是MvM的特例
- 编码设计需要满足: 任意两个类别的编码距离尽可能大
- 具有纠错能力,可容忍部分分类器出错
- 需要特殊设计编码矩阵

- 停用类(0)表示该分类器不考虑该类别样本

方法9: Sigmoid函数 / Sigmoid Function

缩写: 无
页码: Lecture 3-1 p.29

解决问题:

- 任务类型: 激活函数,用于将任意实数映射到(0,1)区间
- 适用场景: 逻辑回归、神经网络等需要概率输出的场景

工作原理:

1. 核心思想: 将线性输出转换为概率值
2. 函数形式: g(z) = 1/(1 + e^(-z))
3. 特性: S形曲线,连续可导,输出范围(0,1)
4. 导数: g'(z) = g(z)(1 - g(z))

重要公式/概念:

- Sigmoid函数: g(z) = 1/(1 + e^(-z)) = e^z/(1 + e^z)
- 输出范围: 0 < g(z) < 1
- 对称性: g(-z) = 1 - g(z)
- 当z→∞时, g(z)→1; 当z→-∞时, g(z)→0

关键要点:

- 连续可导,可用于梯度下降
- 输出可解释为概率
- 梯度消失问题: 当|z|很大时,梯度接近0
- 常用于二分类问题

评价指标

指标1: 混淆矩阵 / Confusion Matrix

页码: Lecture 3-2 p.38-40

解决问题:

- 评估分类模型性能的基础工具
- 适用于二分类和多分类问题

核心概念:

- **TP (True Positive, 真正例)**: 预测为正,实际为正
- **FP (False Positive, 假正例)**: 预测为正,实际为负(第一类错误)
- **FN (False Negative, 假负例)**: 预测为负,实际为正(第二类错误)
- **TN (True Negative, 真负例)**: 预测为负,实际为负

二分类混淆矩阵:

多分类混淆矩阵:

- 行表示实际类别

- 列表示预测类别
- 对角线元素表示正确分类
- 非对角线元素表示错误分类

指标2: 查准率/精确率 / Precision

缩写: P
页码: Lecture 3-2 p.38

解决问题:

- 衡量预测为正例的样本中,真正为正例的比例

重要公式:

- P = TP / (TP + FP)
- 含义: 在所有预测为正的样本中,有多少是真正例

关键要点:

- 关注预测结果的准确性
- 适用于关注"查准"的场景(如推荐系统)
- 值域: [0, 1], 越大越好

指标3: 查全率/召回率 / Recall

缩写: R, TPR (True Positive Rate), Sensitivity
页码: Lecture 3-2 p.38

解决问题:

- 衡量所有正例中,被正确预测为正例的比例

重要公式:

- R = TP / (TP + FN)
- 含义: 在所有真正例中,有多少被预测出来

关键要点:

- 关注模型的覆盖能力
- 适用于关注"查全"的场景(如疾病诊断)
- 值域: [0, 1], 越大越好

指标4: F1分数 / F1 Score

缩写: F1
页码: Lecture 3-2 p.38

解决问题:

- 综合考虑查准率和查全率的调和平均数

重要公式:

- F1 = 2×P×R / (P+R) = 2×TP / (样本总数 + TP - TN)
- 调和平均: 1/F1 = 1/2 × (1/P + 1/R)

关键要点:

- 平衡查准率和查全率
- 当P和R都高时,F1才高
- 值域: [0, 1], 越大越好
- 适用于不平衡数据集

指标5: 准确率 / Accuracy

页码: Lecture 3-2 p.40

解决问题:

- 衡量所有样本中,被正确分类的比例

重要公式:

- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- 对于多分类: Accuracy = 对角线元素之和 / 总样本数

关键要点:

- 最常用的评价指标
- 对不平衡数据集不适用
- 值域: [0, 1], 越大越好

多分类示例计算 (Lecture 3-2 p.40):

- 对于类别A: Precision_A = 15/24, Recall_A = 15/20
- 总准确率 = (15+15+45)/100 = 0.75

指标6: ROC曲线与AUC / ROC Curve & AUC

缩写: ROC (Receiver Operating Characteristic), AUC (Area Under Curve)
页码: Lecture 3-2 p.41

解决问题:

- 评估分类器在不同阈值下的性能
- 衡量模型整体分类能力

核心概念:

- **TPR (True Positive Rate, 真正例率)**: TPR = TP/(TP+FN), 即召回率
- **FPR (False Positive Rate, 假正例率)**: FPR = FP/(TN+FP)
- **ROC曲线**: 以FPR为横轴,TPR为纵轴,绘制不同阈值下的点
- **AUC**: ROC曲线下的面积

关键要点:

- ROC曲线越靠近左上角,性能越好
- AUC值域: [0, 1], AUC=1表示完美分类器
- AUC=0.5表示随机猜测
- 不受类别不平衡影响
- 用于选择最优阈值

概念对照与易混点

1. Linear Regression vs Logistic Regression

核心区别:

- Linear Regression: 输出连续值,用于回归问题,值域(-∞, +∞)
- Logistic Regression: 输出概率值,用于分类问题,值域(0, 1)
- Logistic Regression使用Sigmoid函数将线性输出转换为概率

2. Naive Bayes vs LDA

核心区别:

- Naive Bayes: 假设特征条件独立,基于贝叶斯定理,适用于离散和连续特征
- LDA: 假设各类数据服从多元正态分布且协方差相同,本质也是贝叶斯方法的特例
- LDA同时可用于降维,Naive Bayes不能降维
- LDA决策边界是线性的,Naive Bayes可以是非线性的

3. OvO vs OvR vs MvM

核心区别:

- OvO: N(N-1)/2个分类器,训练集小,分类器多
- OvR: N个分类器,训练集大,可能类别不平衡
- MvM: M个分类器(M自定),具有纠错能力,需设计编码矩阵
- OvO和OvR都是MvM的特例

4. Precision vs Recall

核心区别:

- Precision: 关注预测为正的准确性,"我预测的正样本有多少是对的"
- Recall: 关注覆盖的完整性,"真正的正样本我找到了多少"
- 两者通常存在权衡(trade-off)
- F1 Score是两者的调和平均,综合评价

5. Sigmoid vs Softmax

核心区别:

- Sigmoid: 用于二分类,将实数映射到(0,1)
- Softmax: 用于多分类,将向量映射到概率分布,各元素和为1
- Softmax是Sigmoid在多分类上的推广

6. 概率术语辨析

- **先验概率 (Prior)**: P(Y=k), 类别k的初始概率
- **后验概率 (Posterior)**: P(Y=k|X), 给定观测后类别k的概率
- **似然 (Likelihood)**: P(X|Y=k), 给定类别k后观测到X的概率
- **几率 (Odds)**: P(Y=1)/(1-P(Y=1)), 正例与反例概率之比
- **对数几率 (Logit)**: log(P(Y=1)/(1-P(Y=1))), 几率的对数

7. 分类问题类型

- **二分类 (Binary Classification)**: $y \in \{0,1\}$, 两个类别
- **多分类 (Multi-class Classification)**: $y \in \{C_1, C_2, ..., C_n\}$, N>2个类别
- **多标签 (Multi-label)**: 每个样本可同时属于多个类别(本讲座未涉及)

8. 为什么线性回归不适合分类

1. **输出范围问题**: LR输出$(-\infty, +\infty)$, 分类需要{0,1}或概率[0,1]
2. **离群点敏感**: 新增离群点会显著改变决策边界
3. **多分类编码问题**: 类别编码(如0,1,2)暗含了顺序关系,影响结果
4. **损失函数不合理**: 平方损失对分类任务不是最优选择

关键算法流程

Logistic Regression训练流程

1. 初始化参数$\beta_0, \beta_1, ..., \beta_p$
2. 对每个样本计算预测概率: $P(Y=1|X) = 1/(1+e^{-(\beta_0+\beta_1X_1+...+\beta_pX_p)})$
3. 构建似然函数: $L(\beta) = \Pi^n_{i=1} P(Y=y_i)$
4. 使用MLE或梯度下降最大化对数似然: $\max \Sigma^n_{i=1} \log P(Y=y_i)$
5. 更新参数直到收敛

Naive Bayes分类流程

1. 统计训练集中各类别的先验概率: $\pi_k = n_k/n$
2. 对每个特征,估计条件概率: $P(X_j=x_j|Y=k)$
    - 离散特征: 统计频率
    - 连续特征: 假设分布(如正态分布)并估计参数
3. 对新样本x,计算后验概率: $P(Y=k|X) \propto \pi_k \times \Pi_j P(X_j|Y=k)$
4. 返回后验概率最大的类别: $\hat{y} = \text{argmax}_k P(Y=k|X)$

LDA分类流程(概率视角)

1. 估计每类的先验概率: $\pi_k = n_k/n$
2. 估计每类的均值: $\mu_k = (1/n_k) \Sigma_{i:y_i=k} x_i$
3. 估计共享协方差: $\sigma^2 = (1/n) \Sigma^K_{k=1} \Sigma_{i:y_i=k} (x_{ik}-\mu_k)^2$
4. 对新样本x,计算判别函数: $\delta_k(x) = x \cdot \mu_k/\sigma^2 - \mu_k^2/(2\sigma^2) + \log(\pi_k)$
5. 返回判别函数值最大的类别: $\hat{y} = \text{argmax}_k \delta_k(x)$

ECOC多分类流程

1. **编码**: 设计N×M编码矩阵,每行代表一个类别的M位编码
2. **训练**: 根据编码矩阵的每一列,构建二分类训练集,训练M个分类器
3. **预测**: M个分类器对测试样本预测,得到M位预测编码
4. **解码**: 计算预测编码与各类别编码的海明距离,返回距离最小的类别

Python实现要点 (sklearn)

逻辑回归

朴素贝叶斯

线性判别分析

k近邻

评价指标

考试重点提示

必须记住的公式

1. **Logistic Regression**: $P(Y=1) = 1/(1 + e^{-(\beta_0+\beta_1X)})$
2. **Logit**: $\log(P(Y=1)/(1-P(Y=1))) = \beta_0 + \beta_1X$
3. **Naive Bayes**: $P(Y=k|X) = \pi_k \times \Pi_j P(X_j|Y=k) / P(X)$
4. **LDA判别函数**: $\delta_k(x) = x\cdot\mu_k/\sigma^2 - \mu_k^2/(2\sigma^2) + \log(\pi_k)$
5. **Precision**: $P = TP/(TP+FP)$
6. **Recall**: $R = TP/(TP+FN)$
7. **F1 Score**: $F1 = 2PR/(P+R)$

必须理解的概念

1. 为什么线性回归不适合分类(3个原因)
2. Sigmoid函数的作用和性质
3. Naive Bayes的条件独立性假设
4. LDA的几何意义和概率意义
5. OvO, OvR, MvM的区别和应用场景
6. Precision和Recall的权衡
7. ROC曲线的含义

典型考题类型

1. 给定数据计算逻辑回归预测概率
2. 使用Naive Bayes进行分类(给定条件概率表)
3. 计算混淆矩阵并求Precision, Recall, F1
4. 比较不同分类方法的优缺点
5. 选择合适的多分类策略
6. 解释模型输出(概率、几率、对数几率)

易错点

1. Logistic Regression输出的是概率,不是类别
2. Naive Bayes假设条件独立,不是特征独立
3. LDA假设各类协方差相同
4. OvO产生$N(N-1)/2$个分类器,不是N^2
5. Precision = TP/(TP+FP), 不要和Recall混淆
6. F1是调和平均,不是算术平均

7. 多分类混淆矩阵中,对角线是TP,同行非对角是FN,同列非对角是FP

最后更新: 2026-01-06

覆盖内容: ML-Lecture-3-1 (分类基础、逻辑回归) + ML-Lecture-3-2 (朴素贝叶斯、LDA、评价指标)

页码范围: Lecture 3-1 (p.1-40), Lecture 3-2 (p.1-41)

机器学习讲座（Lecture 1 & 2）- 开卷考试复习资料

快速索引

按任务分类

- **回归（Regression）**：简单线性回归(SLR)、多元线性回归(MLR)、k近邻回归(kNN Regression)
- **分类（Classification）**：k近邻分类(kNN Classification)
- **优化方法**: 梯度下降(GD)、批量梯度下降(BGD)、随机梯度下降(SGD)、小批量梯度下降(MBGD)
- **参数估计**: 普通最小二乘法(OLS)

按缩写字母排序

- **AUC**: Area Under ROC Curve（ROC曲线下面积）
- **BGD**: Batch Gradient Descent（批量梯度下降）
- **GD**: Gradient Descent（梯度下降）
- **kNN**: k-Nearest Neighbor（k近邻）
- **MBGD**: Mini-Batch Gradient Descent（小批量梯度下降）
- **MLR**: Multiple Linear Regression（多元线性回归）
- **MSE**: Mean Square Error（均方误差）
- **OLS**: Ordinary Least Square（普通最小二乘法）
- **ROC**: Receiver Operating Characteristic（受试者工作特征曲线）
- **RSS**: Residual Sum of Square（残差平方和）
- **SE**: Standard Error（标准误差）
- **SGD**: Stochastic Gradient Descent（随机梯度下降）
- **SLR**: Simple Linear Regression（简单线性回归）

详细方法总结

方法1：简单线性回归 / Simple Linear Regression

缩写： SLR

页码： Lecture 2, p.7-26

解决问题：

- **任务类型**: 回归问题（Regression）
- **典型输入输出**: 输入单一预测变量X，输出连续数值Y
- **适用场景**: 根据单一预测变量预测定量响应变量，如根据广告费预测销量

工作原理：

1. **核心思想**: 假设响应变量Y与预测变量X之间存在线性关系，通过特征的线性组合来预测

2. **模型形式**: $Y \approx \beta_0 + \beta_1 X$，其中$\beta_0$是截距(Intercept)，$\beta_1$ 是斜率(Slope/Gradient)
3. **参数估计**: 通过最小化损失函数来估计参数β_0和β_1
4. **优化目标**: 使预测值与真实值之间的误差最小化，找到最接近所有数据点的直线

重要公式/概念:

- **模型公式**: $Y = \beta_0 + \beta_1 X + \varepsilon$
- **损失函数**: $Loss(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - f(x_i))^2$
- **残差**: $e_i = y_i - \hat{y}_i$ (第i个观测值与预测值的差)
- **残差平方和(RSS)**: $RSS = \sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_i))^2$
- **最小二乘估计**:
  - $\beta_0 = \bar{y} - \beta_1 \bar{x}$ (其中$\bar{x}$和$\bar{y}$是样本均值)
- **标准误差(SE)**: $Var(\hat{\mu}) = SE(\hat{\mu})^2 = \frac{\sigma^2}{n}$
- **95%置信区间**: $\hat{\beta}_1 \pm 2 \cdot SE(\hat{\beta}_1)$

关键要点:

- **闭式解优点**: 一步到位，直接代入数据计算得到最优参数
- **闭式解局限**: 计算复杂度高（高维数据不可行）、仅适用于凸且可微的简单损失函数
- **术语**: Feature(特征)、Predictor(预测变量)、Target(目标变量)、Response(响应变量)、Coefficient(系数)、Parameter(参数)
- **模型评估**: 观测值越多，标准误差越小，估计越准确

方法2：多元线性回归 / Multiple Linear Regression

缩写: MLR
页码: Lecture 2, p.27-32

解决问题:

- **任务类型**: 回归问题（Regression）
- **典型输入输出**: 输入p个不同的预测变量$X_1, X_2, \ldots, X_p$，输出连续数值Y
- **适用场景**: 根据多个预测变量预测响应变量，如根据TV、Radio、Newspaper三种广告费预测销量

工作原理:

1. **核心思想**: 在单个模型中为每个预测变量设置独立的斜率系数
2. **模型形式**: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$
3. **优势**: 避免了为每个变量建立单独回归方程的问题，能同时考虑所有预测变量的影响
4. **参数估计**: 通过最小化残差平方和来估计所有参数

重要公式/概念:

- **一般形式**: $f(X_1, X_2, \ldots, X_p) = \beta_0 + \sum_{i=1}^{p} \beta_i X_i$
- **残差平方和(RSS)**: $RSS = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_{i1} - \beta_2 x_{i2} - \ldots - \beta_p x_{ip})^2$
- **优化目标**: $\min_{\beta_0, \beta_1, \ldots, \beta_p \in \mathbb{R}} J(\beta_0, \beta_1, \ldots, \beta_p) = \min \sum_{i=1}^{n} (y_i - (\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij}))^2$

关键要点:

- **相比简单线性回归**: 能处理多个预测变量，避免忽略其他媒介的影响
- **数据结构**: p维特征向量，每个样本有p个特征值和1个响应值
- **求解方法**: 通过偏导数为0求解极值问题（类似简单线性回归）

方法3：梯度下降 / Gradient Descent

缩写： GD, BGD, SGD, MBGD

页码： Lecture 2, p.17-21

解决问题：

- **任务类型**：优化算法（用于各种机器学习模型的参数估计）
- **典型输入输出**：输入损失函数和初始参数，输出最优参数值
- **适用场景**：当无法获得闭式解或闭式解计算复杂度过高时，通过迭代方式优化参数

工作原理：

1. **核心思想**：沿着梯度的反方向移动，逐步找到损失函数的最小值
2. **关键步骤**：
    - 找到合适的方向（梯度的负方向）
    - 找到合适的步长（学习率α）
    - 按照方向和步长更新参数
3. **更新规则**：$\beta_j \leftarrow \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$（同时更新所有参数）
4. **迭代过程**：重复更新直到收敛

重要公式/概念：

- **参数更新**：$\beta_j \leftarrow \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$，for $j = 0, 1, \ldots, m$
- **学习率**α：控制模型参数在训练过程中的更新幅度

关键要点：

- **批量梯度下降(BGD)**：每一步使用所有训练样本计算梯度
- **随机梯度下降(SGD)**：每一步使用一个样本，计算后立即更新参数
- **小批量梯度下降(MBGD)**：每一步使用一定批量的训练样本
- **学习率选择**：学习率过大可能导致震荡，过小导致收敛缓慢
- **适用性**：适用于高维数据和复杂损失函数，不受闭式解限制

方法4：k近邻 / k-Nearest Neighbor

缩写： kNN

页码： Lecture 2, p.34-41

解决问题：

- **任务类型**：分类问题（Classification）和回归问题（Regression）
- **典型输入输出**：输入样本特征，输出类别标签（分类）或数值（回归）
- **适用场景**：根据特征空间中最相似的k个样本进行预测，如租金预测、图像分类

工作原理：

1. **核心思想**：如果一个样本在特征空间中与k个实例最为相似，那么这k个实例中大多数属于哪个类别，该样本也属于这个类别
2. **关键步骤**：
    - 计算给定样本与训练集中每个样本的距离d
    - 选择k值（k个最近邻居的数量）
    - 分类问题：多数投票（Voting），取邻居标签的多数

    - 回归问题：求平均（Average），取邻居值的平均值或加权平均
3. **无训练阶段**: 仅存储训练数据，不进行显式训练（懒惰学习 Lazy Learning）
4. **预测阶段**: 计算距离后实时预测

重要公式/概念：

- **欧氏距离(Euclidean Distance)**: 欧几里得空间中两点间线段的长度，m维空间中两点的真实距离
- **曼哈顿距离(Manhattan Distance)**: 两点在各坐标轴投影长度之和，也称城市街区距离(City Block Distance)

关键要点：

- **参数选择**: k值的选择至关重要（k=3时可能属于某类，k=5时可能属于另一类）
- **非参数方法**: 不假设特定的函数形式
- **监督学习**: 需要带标签的训练数据
- **距离度量**: 欧氏距离和曼哈顿距离是常用选择
- **懒惰学习**: 训练时仅存储数据，预测时才计算
- **回归示例**: 伦敦租房价格预测，根据k个最近邻的租金计算平均值

方法5：普通最小二乘法 / Ordinary Least Square

缩写： OLS

页码： Lecture 2, p.14-16

解决问题：

- **任务类型**: 参数估计方法（用于线性回归）
- **典型输入输出**: 输入训练数据，输出参数的最优估计值
- **适用场景**: 线性回归模型中系数的估计

工作原理：

1. **核心思想**: 通过求解极值问题（导数为0的位置）找到使残差平方和最小的参数值
2. **优化转化**: 将最小化问题转化为求偏导数为0的方程组
3. **求解过程**:
    - 对损失函数$J(\beta_0, \beta_1)$分别对β_0和β_1求偏导
    - 令$\frac{\partial J}{\partial \beta_0} = 0$和$\frac{\partial J}{\partial \beta_1} = 0$
    - 求解方程组得到闭式解
4. **结果**: 得到参数(β_0, β_1)的OLS估计量

重要公式/概念：

- **目标函数**: $\min_{\beta_0, \beta_1 \in \mathbb{R}} J(\beta_0, \beta_1)$
- **闭式解(Closed-form Solution)**:

关键要点：

- **闭式解优点**: 一步到位，直接计算得到最优参数
- **闭式解局限**:
    - 计算复杂度高（高维数据下不可行）
    - 仅适用于凸且可微的简单损失函数
- **几何意义**: $J(\beta_0, \beta_1)$关于每个参数都是二次函数，存在唯一最小值

核心概念与术语

机器学习类型 / Types of Machine Learning

监督学习 / Supervised Learning

- **定义**: 给定训练数据及其期望输出（有标签）
- **子类型**:
    - **回归(Regression)**: 预测连续数值标签，如房价预测、股市趋势
    - **分类(Classification)**: 预测离散类别标签，如T恤尺码、鸢尾花种类
- **典型算法**: KNN、线性模型、决策树、朴素贝叶斯、逻辑回归、SVM、神经网络、XGBoost、LightGBM

无监督学习 / Unsupervised Learning

- **定义**: 给定训练数据（无期望输出，无标签）
- **目标**: 发现数据背后的隐藏结构
- **子类型**:
    - **聚类(Clustering)**: 发现数据的分组结构
    - **降维算法(Dimensionality Reduction)**: 发现数据的低维表示
- **典型算法**: 聚类算法、EM算法、降维算法

半监督学习 / Semi-supervised Learning

- **定义**: 给定训练数据及少量期望输出

强化学习 / Reinforcement Learning

- **定义**: 学习系统（agent）与环境交互，学习最大化奖励信号
- **特点**: 序列决策问题
- **典型算法**: 马尔科夫决策方法

参数化方法

参数化方法意味着我们在使用数据寻找 f 之前，先假设一个特定的 f 函数形式。例如，线性模型是一种参数化方法，它假设

$$f(X_1,\ldots,X_p) = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p.$$

非参数化方法

非参数化方法意味着我们在使用数据寻找 f 之前，不假设一个特定的 f 函数形式。例如，树模型是一种非参数化方法

模型评估方法 / Model Evaluation Methods

页码： Lecture 1, p.43-48

留出法 / Hold-out

- **方法**: 将数据集D分为训练集S和测试集T
- **关键要点**:
    - 采用**分层采样(Stratified Sampling)**保持数据分布一致性不同划分导致不同训练/测试集
    - 多次随机划分避免偶然性
    - 训练集通常占2/3到4/5
    - 训练集太小导致评估结果不稳定，测试集太小降低评估保真度

交叉验证法 / Cross Validation

- **方法**: 将数据集D分为k个子集，k-1个作训练集，1个作测试集
- **k折交叉验证(k-fold Cross Validation)**: 进行k次训练和测试
- **关键要点**:
  - 从D中采用分层采样
  - 不同划分导致不同训练/测试集
  - 减少因数据划分引起的性能估计偏差

自助法 / Bootstrapping

- **方法**: 自助采样(Bootstrap Sampling)
- **优点**:
  - 对数据集较小、难以划分训练/测试集时很有效
  - 从初始数据集产生多个不同训练集，可应用于集成学习
- **缺点**: 改变了初始数据分布，引入估计偏差

模型性能度量 / Model Performance Metrics

回归问题度量

给定 $D = \{(x_1, y_1), (x_2, y_2), \cdots (x_n, y_n)\}$，其中 y_i 是真实值

均方误差 / Mean Square Error (MSE)

- **公式**: $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2$
- **特点**: 越小越好

决定系数 / R^2

- **公式**: $R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$，其中 $= \frac{1}{n}\sum_{i=1}^{n} y_i$
- **特点**: 越大越好，反映模型拟合数据的好坏程度
- **含义**:

$R^2 = 1$：模型完美拟合数据，所有数据点都被模型准确预测（理想情况，但几乎不可能）。

$R^2 = 0$：模型的预测能力与直接使用均值预测的效果一样，说明模型没有学习到有效信息。

$R^2 < 0$：模型比简单的均值预测还要差，可能是模型选择不当或过拟合/欠拟合所致。

分类问题度量

错误率 / Error Rate

- **定义**: 所有预测中错误的比例

精度 / Accuracy

- **公式**: $Accuracy = 1 - Error\ Rate$

查准率 / Precision

- **公式**: $P = \frac{TP}{TP+FP}$
- **含义**: 预测为正例的样本中真正为正例的比例

查全率 / Recall

- **公式**: $R = \frac{TP}{TP+FN}$

- **含义**: 真实正例中被正确预测的比例

F1分数 / F1 Score

- **公式**: $F1 = \frac{2 \times P \times R}{P+R} = \frac{2 \times TP}{2 \times TP + FP + FN}$
- **含义**: 查准率和查全率的调和平均

ROC曲线 / ROC Curve

- **全称**: Receiver Operating Characteristic Curve（受试者工作特征曲线）
- **横轴**: $FPR = \frac{FP}{TN+FP}$（假正例率）
- **纵轴**: $TPR = \frac{TP}{TP+FN}$（真正例率）

AUC / Area Under ROC Curve

- **定义**: ROC曲线下的面积
- **特点**: 越大越好，反映分类器整体性能

模型准确性相关概念

误差类型

训练误差 / Training Error (Empirical Error)

- **定义**: 模型在训练数据上产生的误差
- **别名**: 经验误差

泛化误差 / Generalization Error

- **定义**: 模型在新的、未见过的数据（测试数据）上产生的误差
- **目标**: 我们希望模型的泛化误差尽可能小

拟合问题

过拟合 / Overfitting

- **定义**: 模型过于复杂，记住了训练数据而非学习一般模式
- **表现**: 训练误差很低，但测试误差很高
- **原因**: 模型复杂度过高，训练数据集样本单一，训练数据中噪声干扰过大

如何防止过拟合?

1. 获取和使用更多的数据（数据集增强）
2. 采用合适的模型（控制模型的复杂度）
3. 降低特征的数量
4. L1 / L2 正则化
5. Dropout
6. 在训练过程中每次按一定的概率随机地“删除”一部分隐藏单元（神经元）。
7. Early stopping（提前终止）
8. Early stopping是一种迭代次数截断的方法来防止过拟合的方法，即在模型对训练数据集迭代收敛之前停止迭代来防止过拟合。

欠拟合 / Underfitting

- **定义**: 模型过于简单，无法捕捉数据的潜在模式

- **表现**: 训练误差高，测试误差也高
- **原因**: 模型复杂度过低

标准误差 / Standard Error (SE)

- **定义**: 告诉我们估计值偏离真实值的平均程度
- **公式**: $Var(\hat{\mu}) = SE(\hat{\mu})^2 = \frac{\sigma^2}{n}$
- **特点**: 观测值越多(n越大)，标准误差越小，估计越准确

置信区间 / Confidence Interval

- **定义**: 以一定概率包含真实参数值的区间
- **95%置信区间**: 约95%的可能性会包含参数的真实值
- **影响因素**:
  - 置信水平越高，临界值越大，区间越宽
  - SE越小（样本量越大或数据越集中），区间越窄

概念对照与易混点

参数方法 vs 非参数方法

页码： Lecture 1, p.33

- **参数方法(Parametric Methods)**:
  - 在使用数据估计函数f之前，假设特定的函数形式
  - 例子：线性模型假设$f(X_1, \ldots, X_p) = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p$
- **非参数方法(Non-parametric Methods)**:
  - 不假设特定的函数形式
  - 例子：决策树模型、kNN算法

回归 vs 分类

页码： Lecture 2, p.5-6; Lecture 1, p.19-20

- **回归(Regression)**:
  - 预测连续标签(Continuous labels)
  - 输出是实数值
  - 例子：房价预测、股市趋势预测
- **分类(Classification)**:
  - 预测离散标签(Discrete labels)
  - 输出是类别
  - 例子：T恤尺码分类、鸢尾花种类识别

闭式解 vs 梯度下降

页码： Lecture 2, p.15-21

- **闭式解(Closed-form Solution / OLS)**:
  - 优点：一步到位，直接计算得到最优参数
  - 缺点：计算复杂度高（高维不可行）、仅适用于凸且可微的简单损失函数

- **梯度下降(Gradient Descent)**:
  - 优点：适用于高维数据和复杂损失函数
  - 缺点：需要选择学习率、需要迭代多次、可能收敛到局部最优

BGD vs SGD vs MBGD

页码： Lecture 2, p.21

- **批量梯度下降(BGD)**:
  - 每一步使用所有训练样本
  - 收敛稳定但速度慢
- **随机梯度下降(SGD)**:
  - 每一步使用一个样本，立即更新参数
  - 速度快但波动大
- **小批量梯度下降(MBGD)**:
  - 每一步使用一定批量的样本
  - 平衡了速度和稳定性

预测 vs 推断

页码： Lecture 1, p.32

- **预测(Prediction)**:
  - 目标：根据给定的$X = x$预测Y的值
  - 预测值：$\hat{Y} = f(x)$
- **推断(Inference)**:
  - 目标：理解X和Y之间的关系
  - 问题：哪些预测变量真正与响应变量相关？关系的函数形式是什么？

欧氏距离 vs 曼哈顿距离

页码： Lecture 2, p.35-36

- **欧氏距离(Euclidean Distance)**:
  - 两点间线段的长度
  - m维空间中两点的真实距离
- **曼哈顿距离(Manhattan Distance)**:
  - 投影到坐标轴的长度之和
  - 也称城市街区距离(City Block Distance)
  - 像在城市街区中只能沿街道行走的距离

机器学习流程

页码： Lecture 1, p.24

标准流程

1. **数据收集 / Data Collection**
   - 获取训练数据

2. **数据预处理 / Data Preprocessing**
    - 数据清洗
    - 特征选择
3. **特征提取 / Feature Extraction**
    - 从原始数据中提取有用特征
4. **模型训练与评估 / Model Training and Evaluation**
    - 选择算法模型
    - 参数优化
    - 结果预测
    - 性能评估

"No Free Lunch" 定理

页码： Lecture 1, p.34, 37

- 没有一个算法在所有问题上都表现最好
- 需要根据具体问题选择合适的算法
- 模型选择需要权衡复杂度和性能

重要术语中英对照

中文	英文	说明
特征	Feature	输入变量
预测变量	Predictor	用于预测的变量
输入变量	Input Variable	模型的输入
目标变量	Target	要预测的变量
响应变量	Response	模型的输出
输出变量	Output Variable	预测结果
系数	Coefficient	模型参数
参数	Parameter	模型参数
斜率	Slope	线性关系的倾斜度
梯度	Gradient	函数的变化率
截距	Intercept	直线与y轴的交点
残差	Residual	观测值与预测值的差
损失函数	Loss Function	衡量预测误差的函数
学习率	Learning Rate	控制参数更新幅度
样本均值	Sample Mean	所有样本的平均值

快速记忆要点

线性回归核心

1. **目标**: 找到最接近所有数据点的直线/平面/超平面
2. **方法**: 最小化残差平方和(RSS)
3. **求解**: 闭式解(OLS)或梯度下降(GD)
4. **评估**: MSE、R^2

kNN核心

1. **步骤**: 计算距离 → 选择k → 投票(分类)或求均值(回归)
2. **特点**: 无训练阶段，懒惰学习
3. **关键**: k值选择、距离度量

模型评估核心

1. **方法**: 留出法、交叉验证、自助法
2. **原则**: 保持数据分布一致性（分层采样）
3. **目标**: 降低泛化误差

性能度量核心

1. **回归**: MSE（越小越好）、R^2（越大越好）
2. **分类**: Precision、Recall、F1、AUC（越大越好）
3. **权衡**: 过拟合vs欠拟合

考试重点公式速查

简单线性回归

多元线性回归

梯度下降

性能度量

注意事项:

- 所有公式中的求和符号$\sum$默认为$\sum_{i=1}^{n}$
- $\bar{x}$和$\bar{y}$表示样本均值
- TP=真正例, FP=假正例, TN=真负例, FN=假负例
- 考试时注意区分回归和分类的不同度量标准